Double-Spend and Inflation Impossibility for the X-Chain Settlement Spine with a Z-Chain Shielded Pool

A model-level soundness proof, with the implementation obligations it leaves open

Zach Kelling
Lux Industries, Inc.
research@lux.network

Abstract

The X-Chain Settlement Spine (XSS, LP-225) collapses Lux's parallel spendable ledgers into one visible UTXO ledger on X plus one shielded value pool on Z, joined by a shield/unshield bridge. This note proves, *at the model level*, that the XSS state machine makes double-spend and inflation (unbounded mint) impossible *provided* its primitives are implemented to specification, and it states exactly which cryptographic assumptions and proof obligations the implementation must discharge for the model to bind to the deployed system. We formalise the global state as a per-asset value function $\mathsf{V} = \mathsf{S} + \mathsf{L} + \mathsf{C} + \mathsf{P}$ over the spendable, locked, committed, and shielded pools, and give the value-delta of every transition. **Theorem 1** (no double-spend): every UTXO and every shielded note is consumed at most once, from X's total order, the completeness and persistence of the consumed/nullifier sets, and an idempotent consume guard. **Theorem 2** (conservation / no inflation): V is invariant under every transition except a single bounded, schedule-enforced `RewardMint`; in particular shield and unshield are value-neutral because X tracks a scalar `shieldedPoolTotal` that is debited and credited in lockstep. **Theorem 3** (the turnstile backstop, the headline result): *even if the shielded validity-proof system is unsound*—an adversary forges shielded proofs at will—the value extractable from the shielded pool is bounded by `shieldedPoolTotal`, because X independently maintains that scalar and rejects, at execution, any unshield that would drive it negative. This converts the Zcash-class counterfeiting failure (the 2018 BCTV14 soundness flaw, CVE-2019-7167, which permitted *unbounded* counterfeiting of shielded value below the layer at which per-pool monitoring acts) into a bounded, contained failure, by making the pool-total balance a first-class *consensus-layer transition precondition* rather than a monitoring heuristic. We are explicit about scope: **the shield/unshield execution is design-only today** (LP-225; the ZKVM handlers are stubs), so what is proven is the implication "spec-faithful primitives ⇒ safety," not a claim about the current stubbed code. The five assumptions A1–A5 and their discharge sites are tabulated; Theorem 3 is precisely the statement that the safety floor degrades *gracefully* under failure of the strongest assumption (A4).

Contents

1 Scope, and what is and is not claimed **3**

2 The model **3**

2.1 State 3

2.2 Transitions 4

2.3 Execution semantics 6

3 Assumptions and proof obligations **6**

4 Theorem 1: No double-spend **7**

5 Theorem 2: Conservation and no inflation **8**

6 Theorem 3: The turnstile backstop (why XSS > Zcash) **10**
6.1 The Zcash counterfeiting class 10
6.2 The backstop theorem 10

7 Model-level results vs. implementation obligations **11**
7.1 What is proven, and at what layer 11
7.2 The implementation obligations, enumerated 12
7.3 Mapping to the LP-225 validation properties 12

8 Conclusion **13**

1 Scope, and what is and is not claimed

This is a **design-level soundness proof**. It proves a conditional:

> *If* the XSS primitives are implemented faithfully to the LP-225 specification and the assumptions A1–A5 of Section 3 hold, *then* the safety properties "no double-spend" (Theorem 1) and "no inflation" (Theorem 2) hold, and the contingency bound (Theorem 3) holds.

It does *not* claim that the current implementation has these properties. Per LP-225 [1], the shield/unshield *execution* is a stub (`lux/chains/zkvm/handlers.go` returns `"Not implemented"` on the proof-generation and conversion paths), and **no code tracks the cross-pool total** that Theorems 2 and 3 depend on: a search of the ZKVM and precompile trees finds no visible-plus-shielded accounting. The scalar this proof calls `shieldedPoolTotal` is therefore a **[DESIGN-ONLY]** object; establishing and enforcing it is the central new obligation the proof identifies.

What *is* implemented, and what the proof builds on as ground truth, is the X-side conservation primitive and the Z-side note/nullifier objects:

- **[IMPLEMENTED]** X's syntactic conservation rule inputs $\geq$ outputs + fee, per asset, applied to every xvm transaction type through `lux.VerifyTx` (`node/vms/xvm/txs/executor/syntactic_verifier.go`, calling `node/vms/components/lux/transferables.go:209`; the per-asset flow check is `FlowChecker.Verify` in `node/vms/components/lux/flow_checker.go`).
- **[IMPLEMENTED]** the Z note model: `Note` with `Commitment` and `Nullifier`, the `TransactionTypeShield` / `TransactionTypeUnshield` enum members, and persistent spent-nullifier tracking (`node/chains/zkvm/transaction.go:28-29,108-120`; `node/chains/zkvm/nullifier_db.go`, `MarkNullifierSpent`/`IsNullifierSpent`).
- **[IMPLEMENTED]** the commitment and nullifier primitives: a `CommitmentScheme` interface with Poseidon2 (PQ-safe, hash-based) and Pedersen instantiations, and a nullifier derived as Poseidon2.NullifierHash(*nk*, *commitment*, *leafIndex*) (`node/precompile/zk/commitment.go:15-75,156-163`).
- **[IMPLEMENTED]** the two validity verifiers XSS composes at the boundary: P3Q (ML-DSA-65 / FIPS-204 [6], precompile `0x012205`, `node/precompile/p3q/contract.go`; verifier core is `mldsa.VerifySignatureCtx`) and starkfri (STARK/FRI, precompile `0x012220`, magic header `"P3Q1"`, `node/precompile/starkfri/contract.go`).

We mark every model object with its status as we use it. The reader should treat an **[IMPLEMENTED]** citation as "this mechanism exists and the model reuses it," a **[DESIGN-ONLY]** mark as "this mechanism is specified but not yet wired—the proof is conditional on it being wired to spec," and a **[CRYPTO-REVIEW]** mark as "this is a cryptographic obligation gated on sign-off (Section 3)."

2 The model

We mirror the LP-225 model ([1], §2, §5, §9) and make its state machine precise enough to carry value-deltas. We fix a single asset a throughout; all sums are per-asset and all transitions preserve the asset, so the multi-asset statement is the conjunction over assets (the implemented `FlowChecker` is itself per-asset).

2.1 State

Definition 1 (Global state)**.** A state Σ consists of:

1. A finite **UTXO set** U. Each UTXO u has a value $\mathrm{val}(u) \in \mathbb{N}$ (a `uint64` amount in code) and a **state** $\mathrm{st}(u) \in \{\text{SPENDABLE}, \text{LOCKED}, \text{CONSUMED}\}$. **[IMPLEMENTED]** for SPENDABLE/CONSUMED (an xvm UTXO either exists and is spendable, or has been consumed as a transaction input); **[DESIGN-ONLY]** for LOCKED (the `XLock` output type does not exist in xvm today, LP-225 §3).
2. A set of **commitment objects** K (the `XLock` and `XAppCommitment` of LP-225 §3–§3b). Each $\kappa \in K$ carries an amount $\mathrm{amt}(\kappa)$, a target (chain, module) scope, and a Boolean `consumed` flag, set true at most once. **[DESIGN-ONLY]**.
3. A **shielded pool** on Z: a commitment set CM (the note commitment tree), a spent-nullifier set NF, and—this is the new object—a scalar `shieldedPoolTotal`, written $\mathsf{P} \in \mathbb{N}$, *maintained on X*. CM, NF are **[IMPLEMENTED]** (`state_tree.go`, `nullifier_db.go`); P is **[DESIGN-ONLY]** (no code tracks it, §1).

Definition 2 (Pool sums and the value function)**.** For a state Σ, define the per-asset pool sums

$$\begin{aligned}
\mathsf{S}(\Sigma) &= \sum_{\substack{u \in U \\ \mathrm{st}(u)=\text{SPENDABLE}}} \mathrm{val}(u) && \text{(spendable visible value)},\\
\mathsf{L}(\Sigma) &= \sum_{\substack{u \in U \\ \mathrm{st}(u)=\text{LOCKED}}} \mathrm{val}(u) && \text{(value inside live UTXO locks)},\\
\mathsf{C}(\Sigma) &= \sum_{\substack{\kappa \in K \\ \neg\, \kappa.\texttt{consumed}}} \mathrm{amt}(\kappa) && \text{(value inside live commitments)},\\
\mathsf{P}(\Sigma) &= \texttt{shieldedPoolTotal} && \text{(X's scalar mirror of shielded value)}.
\end{aligned}$$

The **total value function** is

$$\mathsf{V}(\Sigma) \;=\; \mathsf{S}(\Sigma) + \mathsf{L}(\Sigma) + \mathsf{C}(\Sigma) + \mathsf{P}(\Sigma).$$

Remark 1 (P is a scalar, not a sum over notes). The decisive design choice is that P is a single scalar X maintains, *not* a quantity X recomputes by summing note values inside the shielded pool. X cannot read note values—they are hidden. X instead tracks the pool total by *bookkeeping the boundary*: every shield credits P by the (public) burned amount, every unshield debits P by the (public) minted amount. The shielded supply $\sum_{\text{live notes}} \mathrm{val}$ is private and lives on Z; the scalar P is its public, X-enforced upper bound. The entire strength of Theorem 3 comes from this: X's P does not trust the proof system to be sound; it trusts only its own arithmetic on public boundary amounts. The *soundness* of the proof system governs whether $\mathsf{P} = \sum_{\text{live notes}} \mathrm{val}$ holds with equality (no *internal* shielded inflation); its *failure* can at worst make the shielded side believe it holds more than P, and that excess is exactly what the turnstile refuses to let out.

2.2 Transitions

A transition $t : \Sigma \to \Sigma'$ is one of the following. For each we give its precondition, its effect, and its value-delta $\Delta\mathsf{V}(t) = \mathsf{V}(\Sigma') - \mathsf{V}(\Sigma)$. We write val of a set of UTXOs for the sum of their values, $f \geq 0$ for the burned fee.

SPEND

Consume spendable inputs $I \subseteq U$, create spendable outputs O. **Precondition (P-conserve):** $\mathrm{val}(I) = \mathrm{val}(O) + f$, and every $u \in I$ is SPENDABLE. **Effect:** each $u \in I$ becomes CONSUMED;

O are added SPENDABLE. The fee f is burned (leaves the asset's circulating value; we account for it below). **[IMPLEMENTED]**: this is exactly `lux.VerifyTx` + consume-once on inputs. $\Delta\mathsf{V} = -(\text{val}(I)) + \text{val}(O) = -f \leq 0$.

LOCK

Consume spendable inputs I; create one live commitment κ with $\text{amt}(\kappa) = \text{val}(I) - f$ (or, in the `XLock` variant where the lock is the sole output and carries the full amount, $\text{amt}(\kappa) = \text{val}(I)$ with $f = 0$). **Precondition:** P-conserve with κ as the output; I all SPENDABLE. **Effect:** $I \to$ CONSUMED; κ live. **[DESIGN-ONLY]** (LP-225 `LockTx`). $\Delta\mathsf{V} = -f \leq 0$ (S down by $\text{val}(I)$, C up by $\text{val}(I) - f$).

SETTLE

Consume a set of live commitments $\{\kappa_i\}$ *gated by an accepted proof*; create spendable outputs O and/or shielded outputs (handled by composing SHIELD, below). **Precondition:** (i) each κ_i is live and shares the one (chain, module) the proof is scoped to; (ii) the proof verifies under `VerifyChainCommitment` (P3Q and/or starkfri); (iii) **settlement conservation** $\sum_i \text{amt}(\kappa_i) = \text{val}(O) + (\text{shielded outputs})$. **Effect:** each κ_i.`consumed` $\leftarrow$ TRUE; O added SPENDABLE; shielded outputs go through SHIELD. **[DESIGN-ONLY]** (LP-225 `SettleTx`); verifiers **[IMPLEMENTED]**. With no shielded leg, $\Delta\mathsf{V} = 0$ (C down, S up, equal). (With a shielded leg the delta is that of the composed SHIELD, i.e. still 0.)

SHIELD

Burn a spendable amount v on X; add a commitment C to CM on Z; credit $\mathsf{P} \leftarrow \mathsf{P} + v$. **Precondition:** a named spendable UTXO (or settlement leg) of value v exists; a validity proof binds C to value exactly v of the asset. **Effect:** the named visible value is consumed (S down by v); $C \in \mathsf{CM}$; $\mathsf{P} \leftarrow \mathsf{P} + v$. **[DESIGN-ONLY]** execution (handler stub); model objects **[IMPLEMENTED]**. $\Delta\mathsf{V} = -v + v = 0$.

UNSHIELD

Spend a live Z note of value v: publish its nullifier N; *debit* $\mathsf{P} \leftarrow \mathsf{P} - v$; mint a spendable UTXO of value v on X. **Precondition (U1–U3):** (U1) $N \notin \mathsf{NF}$; (U2) a validity proof asserts the note was live and committed to value v; (U3) **turnstile floor** $\mathsf{P} \geq v$. **Effect:** N added to NF (the note is nullified); $\mathsf{P} \leftarrow \mathsf{P} - v$; a spendable UTXO of value v added (S up by v). **[DESIGN-ONLY]** execution; NF tracking **[IMPLEMENTED]**; (U3) and P **[DESIGN-ONLY]**. $\Delta\mathsf{V} = -v + v = 0$.

REWARDMINT

The *only* value-increasing transition. Create a spendable amount r where $r = \text{Sched}(h)$ is the amount the deterministic, X-enforced monetary schedule prescribes at height h. **Precondition:** r equals the scheduled amount for h and the per-height issuance has not already been minted. **Effect:** S up by r. **[DESIGN-ONLY]** as an *X-level* primitive (LP-225 §10 flags that P's reward minting today lives on the wrong chain and must move to an X-settled, schedule-gated result). $\Delta\mathsf{V} = +r$, with r a bounded, deterministic function of consensus height.

Remark 2 (On the fee). The fee f is value removed from the asset's circulating supply (burned). It makes SPEND/LOCK value-*decreasing* by f, never increasing. For the conservation theorem we treat the burned fee as a distinguished sink: define the conserved quantity as $\mathsf{V}(\Sigma) + \text{Burned}(\Sigma)$ where Burned accumulates fees. Then $\Delta(\mathsf{V} + \text{Burned}) = 0$ for SPEND/LOCK. Inflation is the property that V never *rises* except by REWARDMINT; fees can only lower it, so they never threaten the no-inflation bound and we fold them into the "$\leq$" direction without further comment. (In code the fee burn

is `fc.Produce(feeAssetID, feeAmount)` in `VerifyTx`: the fee is counted on the produced/output side, so inputs $\geq$ outputs + fee is what `FlowChecker.Verify` enforces.)

2.3 Execution semantics

Definition 3 (Ledger execution)**.** The ledger is the sequence of states $\Sigma_0, \Sigma_1, \ldots$ where Σ_0 is genesis and Σ_{j+1} results from applying the transitions of block j, in the block's order, each only if its precondition holds in the then-current state (a transition whose precondition fails is *rejected*: it is not applied and Σ is unchanged by it). X's consensus fixes a single total order on all blocks and hence on all applied transitions (Assumption A1).

This is the operational content of LP-225's "X owns the total ordering of all asset movements" and "other chains compute, X settles": every state-moving transition is an X transition, applied in X's order, gated by an X-checked precondition. Computing modules (P, D, B, C, Q) produce only *proofs* and *settlement requests*; the value movement is the SETTLE transition on X (LP-225 §2, property S3).

3 Assumptions and proof obligations

The theorems are proven relative to the following assumptions. Each is the implementation contract for one mechanism; each names where it must be discharged and flags what is **[DESIGN-ONLY]** today. The numbering A1–A5 is referenced throughout.

Assumption 1 (Total order and atomic apply)**.** X consensus assigns all applied transitions a single total order, and each transition is applied atomically: its precondition is evaluated against the unique pre-state and, if it holds, its entire effect is applied before the next transition; if it fails, none of its effect is applied. No two conflicting histories are both finalised (no fork double-apply).
Discharge: Lux consensus safety/finality (proven separately: `lean/Consensus/Safety.lean`, `Finality.lean`, BFT quorum intersection; QuasarCert soundness [10]). Within a single chain, transaction application is sequential and atomic (xvm block execution). **[IMPLEMENTED]** (consensus + xvm execution).

Assumption 2 (Consumed-set / nullifier-set completeness and persistence)**.** (i) The set of consumed UTXO references is complete and persistent: a UTXO consumed by any applied transition is, at every later height, recorded as consumed and unavailable as an input. (ii) The spent-nullifier set NF is complete and persistent: a nullifier published by any applied UNSHIELD (or Z-internal spend) is, at every later height, in NF. (iii) Both are checked *before* a consume is permitted.
Discharge: UTXO consume-once is **[IMPLEMENTED]** (xvm input semantics; the DEX atomic rail's `MarkConsumed`/`IsConsumed` persisted set, `node/chains/dexvm/atomic.go`). Nullifier persistence is **[IMPLEMENTED]** (`node/chains/zkvm/nullifier_db.go` writes each spent nullifier to the backing `database.Database`; `IsNullifierSpent` reads it). **Cross-boundary completeness**—that the *same* NF is the arbiter for both X-side unshield and Z-side spend, with no partition admitting two consumptions—is **[CRYPTO-REVIEW]**/**[DESIGN-ONLY]** (LP-225 §10 item 2, S4).

Assumption 3 (Commitment scheme: binding and hiding)**.** The note commitment scheme is computationally *binding* (no efficient adversary opens one commitment C to two distinct $(v, \cdot)$ pairs) and *hiding* (a commitment reveals nothing about v). The nullifier is *deterministically and uniquely* derived from a note (one live note $\Rightarrow$ exactly one nullifier), and *unforgeable* (cannot produce a valid

nullifier for a note one cannot open).
Discharge: `CommitmentScheme` with Poseidon2 (binding under collision-resistance of the Poseidon2 sponge; PQ-safe) and Pedersen (binding under discrete-log; hiding information-theoretically) (`node/precompile/zk/commitment.go:15-75`). Nullifier determinism is **[IMPLEMENTED]** in form ($\mathsf{NullifierHash}(nk, C, leafIndex)$, `commitment.go:156-163`). **Binding/hiding as deployed**, and the unforgeability of the nullifier under the chosen nk derivation, are **[CRYPTO-REVIEW]** (LP-225 §10 item 1; the scheme must be pinned and reviewed).

Assumption 4 (Validity-proof soundness)**.** The shielded validity-proof system (the STARK/FRI AIR verified by starkfri, and/or the P3Q attestation) is *sound*: an accepting proof implies the asserted statement is true (the note was live and committed to value v; the settled state transition was valid and the deltas are exactly the claimed ones), except with negligible probability.
Discharge: starkfri verifies a STARK/FRI proof (`node/precompile/starkfri/contract.go`); P3Q verifies an ML-DSA-65 attestation (`node/precompile/p3q/contract.go`). **The settlement/shield AIR does not exist yet** and must be specified and reviewed; the P3Q attestation's soundness reduces to honesty of the attesting set (a trusted-operator assumption). Both are **[CRYPTO-REVIEW]** (LP-225 §10 items 4–5). **Theorem 3 is precisely the statement that the safety floor survives the failure of this assumption.**

Assumption 5 (Collision-resistant hash)**.** The hash used for UTXO identifiers, commitment-tree nodes, and nullifiers is collision-resistant, so distinct objects have distinct identifiers (no UTXO/note aliasing) except with negligible probability.
Discharge: the hash families in use (Poseidon2 for the shielded side; the xvm UTXO id hashing on the visible side). Standard assumption; the specific instantiations are **[CRYPTO-REVIEW]**-pinned with the commitment scheme.

Remark 3 (Why A4 is special). A1, A2, A5 are discharged by mechanisms that exist and are largely **[IMPLEMENTED]**. A3 is a standard, well-understood primitive obligation. A4 is the *hardest and least settled*: the settlement/shield AIR is not yet written, and the P3Q attestation path is an explicit trusted-operator assumption. A system whose safety *rests entirely* on A4 is exactly a system with the Zcash failure mode: an A4 break is a counterfeiting break. The architectural thesis of XSS, formalised in Theorem 3, is that the turnstile (U3 plus X's independent P) bounds the damage of an A4 failure to P—so A4 governs *privacy-pool internal* correctness and the equality $\mathsf{P} = \sum_{\text{live notes}} \text{val}$, but not the *global* no-inflation floor.

4 Theorem 1: No double-spend

Theorem 1 (No double-spend)**.** *Under A1 (total order, atomic apply), A2 (consumed/nullifier set completeness, persistence, check-before-consume), and A5 (collision resistance), every UTXO and every shielded note can be consumed by at most one applied transition over the entire ledger history. Equivalently: no applied transition consumes a UTXO already* CONSUMED *or a commitment already* `consumed`*, and no applied* UNSHIELD *(or Z-spend) publishes a nullifier already in* NF.

Proof. Fix a UTXO u (the note case is identical with "nullifier N in NF" in place of "$\text{st}(u) =$ CONSUMED", and "commitment κ with κ.`consumed`" for the lock case). By A5, u has a unique identifier, so "the same UTXO" is well defined: two references with the same id are the same object, and no two distinct objects collide.

Suppose for contradiction that two applied transitions t_1, t_2 each consume u. By A1 the applied transitions are totally ordered; without loss of generality t_1 precedes t_2. Consider the moment t_2 is

evaluated. By the atomic-apply clause of A1, t_1's *entire* effect was applied before t_2 is evaluated, and by A2(i) the consume effect of t_1 is recorded and *persistent*: at the height of t_2, u is recorded CONSUMED (resp. $N \in \mathsf{NF}$, resp. $\kappa.\texttt{consumed} = \text{TRUE}$). By A2(iii) the consume guard checks this set *before* permitting the consume: t_2's precondition requires its inputs be SPENDABLE (for SPEND/LOCK), its commitments be live (for SETTLE), or $N \notin \mathsf{NF}$ (for UNSHIELD). Each of these fails for u (resp. κ, N). Hence t_2's precondition is false; by Definition 3 a transition with a false precondition is rejected and applies no effect. This contradicts the assumption that t_2 is an *applied* transition consuming u.

Therefore at most one applied transition consumes u (resp. κ, N). As u was arbitrary, this holds for every UTXO, every commitment, and every note. □ □

Remark 4 (The three pillars are each necessary). The proof uses exactly three facts, and each is load-bearing: (i) *total order + atomicity* (A1) is what lets us speak of "t_1 before t_2" and guarantees t_1 fully landed before t_2 is judged— a fork that finalised two conflicting histories would let both consume u, which is why Theorem 1 *reduces* double-spend safety to consensus safety; (ii) *completeness + persistence* (A2 i,ii) is what makes the record of t_1's consume visible at t_2's height—a consumed-set that forgot, or that lived only in one partition, would not block t_2; (iii) *check-before-consume* (A2 iii) is the idempotent guard—the precondition reads the set and refuses. Drop any one and the proof fails, which is the precise sense in which these are the implementation obligations for double-spend safety. The cross-boundary case (a note both unshielded on X and re-spent on Z) is the statement that A2(ii) holds for a *single* NF shared across the seam; LP-225 §10(2) flags this as the specific **[CRYPTO-REVIEW]** obligation, and it is the hypothesis the theorem makes explicit, not one it discharges.

5 Theorem 2: Conservation and no inflation

Theorem 2 (Conservation / no inflation)**.** *Under A1, the value function* V *of Definition 2 satisfies, for every applied transition* t*,*

$$\Delta\mathsf{V}(t) = \begin{cases} +\,\mathrm{Sched}(h) & t = \textsc{rewardMint at height } h, \\ -\,f \le 0 & t \in \{\textsc{spend}, \textsc{lock}\} \ \textit{(fee } f \ge 0), \\ 0 & t \in \{\textsc{settle}, \textsc{shield}, \textsc{unshield}\}. \end{cases}$$

Consequently, over any run $\Sigma_0 \to \cdots \to \Sigma_n$*,*

$$\mathsf{V}(\Sigma_n) \;=\; \mathsf{V}(\Sigma_0) \;+ \sum_{j:\ t_j = \textsc{rewardMint}} \mathrm{Sched}(h_j) \;- \sum_{j:\ t_j \in \{\textsc{spend},\textsc{lock}\}} f_j.$$

In particular V *never increases except by* REWARDMINT*, and the total increase up to height* H *is exactly* $\sum_{h \le H} \mathrm{Sched}(h)$*, a bounded, deterministic, X-enforced quantity. No transition admits unbounded or out-of-schedule mint.*

Proof. We compute $\Delta\mathsf{V}$ transition-by-transition, using Definition 2 ($\mathsf{V} = \mathsf{S} + \mathsf{L} + \mathsf{C} + \mathsf{P}$) and the effects of Section 2.2. By A1 each applied transition's effect is applied atomically to a unique pre-state, so the deltas compose additively along the run, giving the displayed telescoping sum once the per-transition deltas are established.

SPEND. Inputs I (SPENDABLE) become CONSUMED: S decreases by $\mathrm{val}(I)$. Outputs O added SPENDABLE: S increases by $\mathrm{val}(O)$. The precondition P-conserve forces $\mathrm{val}(I) = \mathrm{val}(O) + f$. Hence $\Delta\mathsf{S} = \mathrm{val}(O) - \mathrm{val}(I) = -f$; $\mathsf{L}, \mathsf{C}, \mathsf{P}$ unchanged. $\Delta\mathsf{V} = -f \le 0$. *This is the implemented invariant:*

`FlowChecker.Verify` rejects unless produced $\leq$ consumed per asset, i.e. $\mathrm{val}(O) + f \leq \mathrm{val}(I)$, so a spend can never make V rise.

LOCK. I (SPENDABLE) $\to$ CONSUMED: S down by $\mathrm{val}(I)$. One commitment κ with $\mathrm{amt}(\kappa) = \mathrm{val}(I) - f$ becomes live: C up by $\mathrm{val}(I) - f$. $\Delta\mathsf{V} = -\mathrm{val}(I) + (\mathrm{val}(I) - f) = -f \leq 0$. (Value moves spendable $\to$ committed, less the burned fee.)

SETTLE (no shielded leg). Commitments $\{\kappa_i\}$ go `consumed`: C down by $\sum_i \mathrm{amt}(\kappa_i)$. Outputs O added SPENDABLE: S up by $\mathrm{val}(O)$. Precondition (iii) settlement-conservation forces $\sum_i \mathrm{amt}(\kappa_i) = \mathrm{val}(O)$. $\Delta\mathsf{V} = -\sum_i \mathrm{amt}(\kappa_i) + \mathrm{val}(O) = 0$. (With a shielded leg, the shielded portion is realised by composing SHIELD, whose delta is 0 by the next case; settlement-conservation then reads $\sum_i \mathrm{amt}(\kappa_i) = \mathrm{val}(O) + v_{\text{shielded}}$, and the total delta is still 0.) X checks the sum directly and does *not* re-execute the module—this is the LP-224 "agree on the result, check the sum" discipline.

SHIELD. A visible v is burned: S down by v. A commitment C is added to CM and $\mathsf{P} \leftarrow \mathsf{P} + v$: P up by v. C, L unchanged. $\Delta\mathsf{V} = -v + v = 0$. This is the turnstile entry: the value does not vanish from V, it moves from the visible term S into the scalar term P. (Note the proof system is *not* invoked to compute the delta: the burned v is a public X amount, and P is credited by that same public v. A4 governs only whether C *inside* the pool is a faithful commitment to v, not the $\Delta\mathsf{V} = 0$ accounting.)

UNSHIELD. A note of value v is nullified and $\mathsf{P} \leftarrow \mathsf{P} - v$: P down by v (and U3 guarantees $\mathsf{P} \geq v$ beforehand, so P stays in $\mathbb{N}$). A spendable UTXO of value v is minted: S up by v. $\Delta\mathsf{V} = -v + v = 0$. The turnstile exit: value moves from P back to S, conserving V. Again the debit uses the *public* minted amount v, matched to the credit on S.

REWARDMINT. A spendable $r = \mathrm{Sched}(h)$ is created: S up by r. No pool is debited. $\Delta\mathsf{V} = +r$. By precondition, r is exactly the schedule's per-height amount and the per-height issuance is minted at most once, so the cumulative increase to height H is $\sum_{h \leq H} \mathrm{Sched}(h)$, bounded and deterministic.

Summing along the run (the deltas compose by A1's atomic apply) gives the displayed equation. Every term except the REWARDMINT sum is ≤ 0, so V never rises except by REWARDMINT; and that rise is the bounded schedule total. □ □

Corollary 3 (Boundary turnstile is value-neutral; visible + shielded = const)**.** *Restricting to runs with no REWARDMINT and no fees (or folding fees into* Burned *per Remark 2), the LP-225 boundary invariant holds: for every height h,*

$$\underbrace{[\,\mathsf{S}(\Sigma_h) + \mathsf{L}(\Sigma_h) + \mathsf{C}(\Sigma_h)\,]}_{\textit{visible}} + \underbrace{\mathsf{P}(\Sigma_h)}_{\textit{shielded (X's scalar)}} = \mathrm{total}(a), \quad \textit{constant in } h.$$

Proof. Immediate from Theorem 2: with no REWARDMINT and no fee sink, every $\Delta\mathsf{V} = 0$, so $\mathsf{V} = \mathsf{S} + \mathsf{L} + \mathsf{C} + \mathsf{P}$ is constant; set $\mathrm{total}(a) := \mathsf{V}(\Sigma_0)$. SHIELD/UNSHIELD shift mass between the visible terms and P without changing the sum. □ □

Remark 5 (What "no inflation" does and does not assert). Theorem 2 is an *accounting* statement about V, which uses the scalar P, *not* the private sum of note values. It proves: no XSS transition raises the total except the scheduled mint. It does *not* by itself prove that the shielded pool's *internal* contents are consistent ($\mathsf{P} = \sum_{\text{live notes}} \mathrm{val}$); that equality is what A4 (proof soundness) buys, and its failure is the subject of Theorem 3. The separation is deliberate and is the whole point: global no-inflation is provable from *arithmetic X enforces on public boundary amounts* (A1 alone, modulo the schedule), independent of the proof system.

6 Theorem 3: The turnstile backstop (why XSS > Zcash)

This is the headline. Theorems 1 and 2 assume the proof system is sound (A4). Theorem 3 *drops* A4 and bounds the damage.

6.1 The Zcash counterfeiting class

In 2018, Ariel Gabizon discovered a soundness flaw in the BCTV14 zk-SNARK construction used by Zcash's original Sprout shielded pool: a spurious element in the proving key allowed an attacker to produce *accepting proofs of false statements*, i.e. to forge a shielded "spend" of value that was never deposited. Disclosed as CVE-2019-7167 and remediated by the 2018 Sapling upgrade, the flaw permitted—in principle—*unbounded, undetectable* counterfeiting of shielded value, because the only thing standing between a forged proof and minted value was the soundness of the proof system itself [4, 5]. Electric Coin Company found no evidence the flaw was exploited.

The structural lesson: in the Sprout design, the proof system's soundness was the *sole* line of defence for the integrity of the shielded supply. A break at that layer is a break of the whole supply. Zcash later operated per-pool value-balance ("turnstile") tracking, and ECC's own analysis is that such tracking *bounds* the damage of an undetected break to the size of the affected pool—but in the relevant era this was a monitoring/alerting mechanism layered *above* the proof system, not a hard per-transaction consensus reject derived from an independently maintained pool total.

Remark 6 (Honest framing of the contrast). We do *not* claim Zcash had no turnstile. We claim a precise structural difference: XSS makes the pool-total balance a **consensus-layer precondition U3 of the unshield transition itself**—an unshield that would drive X's independently maintained P negative is *rejected at execution*, fail-closed, by the same X check that enforces inputs $\geq$ outputs + fee (Theorem 2, the `FlowChecker` discipline). The bound is therefore not a heuristic that detects an anomaly after the fact; it is an invariant that no valid state transition can violate. Where Zcash's defence-in-depth for the *integrity of supply* ultimately rested on A4, XSS's *global no-inflation floor* rests on A1 (X's arithmetic), with A4 demoted to governing only the pool's internal correctness.

6.2 The backstop theorem

Theorem 4 (Turnstile backstop under unsound proofs)**.** *Assume A1, A2, A5, and the turnstile precondition U3 (an* UNSHIELD *of value v requires $\mathsf{P} \geq v$, and atomically sets $\mathsf{P} \leftarrow \mathsf{P} - v$), with P maintained on X by the* SHIELD/UNSHIELD *bookkeeping of Section 2.2.* ***Drop A4 entirely****: allow the adversary to produce accepting shielded validity proofs for arbitrary (including false) statements. Then over any run, the total value ever moved out of the shielded pool into the visible ledger is bounded by the total value ever legitimately moved in:*

$$\sum_{UNSHIELD} v_{out} \;\leq\; \mathsf{P}(\Sigma_0) \;+\; \sum_{SHIELD} v_{in},$$

and at every height $\mathsf{P}(\Sigma_h) \geq 0$. In particular the visible supply $\mathsf{S}+\mathsf{L}+\mathsf{C}$ can never exceed $\mathrm{total}(a) = \mathsf{V}(\Sigma_0) + (\textit{scheduled mint})$*:* ***a shielded proof forgery cannot inflate the visible supply, and cannot extract from the pool more than was put in.***

Proof. Track the scalar P as an integer over the run. It changes only on SHIELD ($\mathsf{P} \leftarrow \mathsf{P} + v_{\text{in}}$) and UNSHIELD ($\mathsf{P} \leftarrow \mathsf{P} - v_{\text{out}}$); these updates are by A1 atomic and applied in a single total order. By precondition U3, an applied UNSHIELD of value v_{out} requires $\mathsf{P} \geq v_{\text{out}}$ in the pre-state; hence after it $\mathsf{P} \geq 0$. SHIELD only increases P. By induction over the totally ordered run, $\mathsf{P}(\Sigma_h) \geq 0$ for all h.

Now sum the updates. After the run,

$$\mathsf{P}(\Sigma_n) = \mathsf{P}(\Sigma_0) + \sum_{\text{SHIELD}} v_{\text{in}} - \sum_{\text{UNSHIELD}} v_{\text{out}} \;\geq\; 0,$$

which rearranges to exactly $\sum_{\text{UNSHIELD}} v_{\text{out}} \leq \mathsf{P}(\Sigma_0) + \sum_{\text{SHIELD}} v_{\text{in}}$.

Crucially, *nothing in this argument used A4.* The bound U3 is a check X performs on its *own* scalar P against the *public* minted amount v_{out}; the adversary's forged proof may convince the shielded side that some note is spendable, but it cannot change the value of X's integer P, and it cannot make X's U3 comparison $\mathsf{P} \geq v_{\text{out}}$ pass when it would otherwise fail. The forgery's power is confined to the shielded pool's internal belief about which notes are live; the moment value attempts to *leave* via UNSHIELD, it meets X's pool-total floor.

Finally, by Theorem 2 every UNSHIELD has $\Delta\mathsf{V} = 0$ *regardless of A4* (its delta is the public $-v_{\text{out}}$ on P plus $+v_{\text{out}}$ on S), so the visible supply rises by exactly what P falls, and P never goes negative; hence the visible supply is bounded by total(a) as stated. □ □

Corollary 5 (Graceful degradation; the failure is contained, not catastrophic)**.** *Under an A4 break, the worst an adversary can achieve is to redistribute* within the shielded pool's P budget*—e.g. unshield value that, inside the pool, "belonged" to another user—stealing up to* P *from honest shielded holders, but* ***minting zero net new supply****. The blast radius of a total proof-soundness failure is therefore at most* P*, the shielded pool total, and is borne as a redistribution among shielded users, never as inflation of the asset. Contrast the Zcash-Sprout class, where the same break is* unbounded *counterfeiting of new supply.*

Proof. By Theorem 4, total unshielded value $\leq \mathsf{P}(\Sigma_0) + \sum v_{\text{in}}$, the value legitimately placed in the pool. An adversary who can forge proofs can direct unshields to itself up to that ceiling, draining at most the pool's contents; it cannot exceed the ceiling, so net new supply is zero. The honest-user loss is bounded by P. □ □

Remark 7 (Where this leaves A4). Theorem 3 does not make A4 unnecessary—A4 is what guarantees that the *distribution* of value inside the pool is correct (your notes are yours; the equality $\mathsf{P} = \sum_{\text{live notes}} \text{val}$ holds), and hence that an honest shielded user is not robbed by a forger. It makes A4 *non-catastrophic*: its failure is a bounded theft within the pool, not an unbounded mint that debases every holder of the asset. This is the precise sense in which the XSS turnstile, as a consensus invariant, is strictly stronger than relying on proof soundness alone, and is the soundness argument for "XSS contains the Zcash counterfeiting class."

7 Model-level results vs. implementation obligations

7.1 What is proven, and at what layer

- **Proven at the model level** (Theorems 1–3, from A1–A5 as stated): given spec-faithful primitives, double-spend is impossible, inflation is impossible beyond the scheduled mint, and a total shielded-proof-soundness failure is contained to the shielded pool total (no visible inflation, bounded intra-pool redistribution). These are theorems about the *state machine*, and they are unconditional given the assumptions.
- **Grounded in implemented code**: the visible-side conservation primitive (inputs ≥ outputs+fee via `lux.VerifyTx`/`FlowChecker`), UTXO consume-once, the Z note/commitment/nullifier objects and persistent NF, the commitment and nullifier primitives, and the P3Q and starkfri verifiers all exist today. Theorem 1's machinery (A1, A2) is largely **[IMPLEMENTED]**.

- **Design-only, and required before the model binds to the system**: the items below. Until they are implemented to spec, the theorems are statements about XSS-as-specified, not about the running code.

7.2 The implementation obligations, enumerated

1. **The scalar P (`shieldedPoolTotal`) must be implemented and X-enforced. [DESIGN-ONLY]**. This is the linchpin of Theorems 2 and 3 and *no code tracks it today* (§1; LP-225 §5, §9b/P2). The shield path must credit it by the public burned amount, the unshield path must debit it by the public minted amount and enforce U3 ($\mathsf{P} \geq v$) as a hard, fail-closed precondition—in the same place and with the same discipline as the existing `FlowChecker` conservation check. Without U3, Theorem 3 is vacuous.

2. **The shield/unshield execution must be implemented. [DESIGN-ONLY]**. The ZKVM handlers are stubs (`node/chains/zkvm/handlers.go`: "Not implemented" / "Would implement shield proof generation"). The model's SHIELD and UNSHIELD transitions are conditional on these existing and performing exactly the burn-credit and nullify-debit-mint effects above.

3. **A2 cross-boundary completeness (S4) must be discharged. [CRYPTO-REVIEW]/[DESIGN-ONLY]**. Theorem 1 assumes a *single* NF arbitrates both X-side unshield and Z-side spend with no partition admitting two consumptions (LP-225 §10 item 2). The persistent NF exists; the cross-seam single-arbiter property must be designed and reviewed.

4. **A3 binding/hiding and nullifier unforgeability must be pinned and reviewed as deployed. [CRYPTO-REVIEW]**. The `CommitmentScheme` interface and the Poseidon2/Pedersen instances exist; the deployed choice and parameters require cryptographer sign-off (LP-225 §10 item 1).

5. **A4 soundness: the settlement/shield AIR must be specified and reviewed. [CRYPTO-REVIEW]**. starkfri verifies a STARK/FRI proof but the settlement-validity AIR does not yet exist (LP-225 §10 items 4–5); the P3Q attestation is an explicit trusted-operator assumption with a policy boundary (when is attestation acceptable vs. when is a STARK mandatory?) to be defined. **Theorem 3 is the reason an A4 gap is survivable, not the reason it can be ignored**: A4 still governs intra-pool correctness (Corollary 5, Remark 7).

6. **REWARDMINT must be moved to an X-level, schedule-gated primitive. [DESIGN-ONLY]**. Today P mints rewards on its own chain (LP-225 §9/S3, §10 item 2), contradicting "X settles." Theorem 2's "only value-increasing transition is bounded by an X-enforced schedule" holds only once issuance is an X-settled, deterministic, per-height-once primitive.

7.3 Mapping to the LP-225 validation properties

The three theorems are the formal core of LP-225's safety properties S1–S5 and validation checklist P1–P8:

This proof	LP-225 property	Status of the underlying mechanism
Thm 1 (no double-spend)	S1, S4, S5; P1, P8	A1/A2 mostly **[IMPLEMENTED]**; S4 seam **[CRYPTO-REVIEW]**
Thm 2 (conservation/no inflation)	S2; P2	visible **[IMPLEMENTED]**; P **[DESIGN-ONLY]**
Thm 3 (turnstile backstop)	S2 shielded term; P2	U3 + P **[DESIGN-ONLY]** (the new obligation)

The single most important deliverable the proof identifies is obligation 1: **implement and X-enforce `shieldedPoolTotal` with the fail-closed U3 check**. It is the mechanism that turns the Zcash-class failure from catastrophic (unbounded mint) into contained (bounded intra-pool redistribution), and it costs one scalar and one comparison, enforced in the same code path that already enforces visible conservation. Everything else in the shielded design (A3, A4) then governs *who* holds the shielded value, never *how much* can leave the pool.

8 Conclusion

We have proven, at the model level, that the XSS unified-settlement model with a Z-Chain shielded pool makes double-spend and inflation impossible under explicit, enumerated assumptions, and—the headline—that even a total failure of the shielded validity-proof system (the assumption that breaks Zcash-class designs) cannot inflate the asset or extract more than the pool holds, because X maintains the shielded pool total as a scalar and refuses, at the consensus layer and fail-closed, any unshield that would drive it negative. The proof is honestly conditional: the shield/unshield execution and the `shieldedPoolTotal` scalar are design-only today (LP-225), and the cryptographic assumptions A3–A4 require sign-off. What the proof establishes is the implication—*spec-faithful primitives ⇒ safety, with graceful degradation under the hardest assumption*—together with the exact contract (obligations 1–6) the implementation must discharge for the running system to inherit the model's guarantees.

References

[1] Lux Industries. *LP-225: XSS — The X-Chain Settlement Spine: One Economic Ledger, Many Computing Chains.* Lux Improvement Proposal, 2026. (Design specification; `~/work/lux/papers/lp-225-xss-x-chain-settlement-spine`.)

[2] S. Nakamoto. *Bitcoin: A Peer-to-Peer Electronic Cash System.* 2008. (The UTXO model and the inputs-equal-outputs conservation rule.)

[3] S. Bowe, A. Gabizon, et al. *Zcash Protocol Specification (Sapling).* 2018. (Shielded notes, commitments, nullifiers, and the spend/output circuit.)

[4] Electric Coin Company. *Zcash Counterfeiting Vulnerability Successfully Remediated.* Disclosure of CVE-2019-7167, the BCTV14 soundness flaw in the Sprout shielded pool (discovered 2018-

03-01 by A. Gabizon, remediated in the Sapling upgrade 2018-10-28). https://electriccoin.co/blog/zcash-counterfeiting-vulnerability-successfully-remediated/, 2019.

[5] Zcash Foundation. *Concerning the Sprout Vulnerability CVE-2019-7167.* https://zfnd.org/concerning-the-sprout-vulnerability-cve-2019-7167/, 2019.

[6] NIST. *FIPS 204: Module-Lattice-Based Digital Signature Standard (ML-DSA).* 2024. (The verifier core of the P3Q precompile, `0x012205`.)

[7] Lux Industries. *LP-218: ZAP P3Q Post-Quantum Rollup.* 2026.

[8] Lux Industries. *LP-221: STARK-FRI Verifier.* 2026. (The starkfri precompile, `0x012220`.)

[9] Lux Industries. *LP-224: D-Chain Deposit/Withdraw Consensus-Wire Activation.* 2026. (The "agree on the result, check the sum" settlement discipline.)

[10] Z. Kelling, Lux Industries. *QuasarCert Soundness: Canonical Formal Specification, Quasar 3.0.* 2025. (X-consensus safety/finality; companion proof `quasar-cert-soundness.tex`.)